

Designated Lands — ArcGIS Pro Edition

Under section 63 of the *Species at Risk Act* (SARA), the Government of Canada must report every 180 days on the protection of Critical Habitat for listed species on non-federal lands. This pipeline supports that obligation for British Columbia by consolidating 40+ provincial land and marine designations into a single unified *Designated Lands* dataset using ArcGIS Pro and arcpy, and then intersecting the result with ECCC's **Critical Habitat Area Final (CHA)** polygons to quantify how much critical habitat is already covered by existing protections, with the option to add a date filter to find if there are any new protections implemented in the within the last date range. Each designation is categorized by the level of restriction it imposes on three industry sectors — forestry, oil & gas, and mining — at five levels: **Protected**, **Full**, **High**, **Medium**, **Low**, and **None**.

This is a re-implementation of the [original designatedlands tool](#) (which used PostgreSQL/PostGIS), but extended in this script with CHA intersection, date-based change detection, and SARA reporting capabilities. This version replaces the database backend with Esri File Geodatabases and arcpy geoprocessing, making it runnable on any workstation with an ArcGIS Pro license. For more info on that dataset's methodology read [here](#).

Methodology

Objective

This project serves two complementary objectives arising from Canada's **Species at Risk Act (SARA)** reporting obligations:

1. **Recurring 180-day report** — Under SARA section 63, the Government of Canada must report every 180 days on the protection of Critical Habitat for listed species. This pipeline produces a date-filtered analysis of newly established or modified designations within a reporting window, intersected with ECCC's Critical Habitat Area (CHA) polygons on non-federal land, quantifying changes in protection coverage since the last report.

2. **Long-term reusable dataset** — Beyond the recurring report, the pipeline produces a comprehensive, province-wide spatial dataset of overlapping land designation or planarized land designations as well as a dataset describing all existing protections that overlap with CHA.

Data Compilation

Forty-two designation layers are compiled from two categories of sources:

- **BC Geographic Warehouse (BCGW)** — The majority of layers are downloaded programmatically from the province's public Web Feature Service (WFS) endpoint. Each layer is defined in `sources_designations.csv` with a BC Data Catalogue URL, an optional CQL query filter to select the relevant subset of features, and an optional date-filter template for change-detection workflows.
- **External / federal sources** — A small number of layers (National Parks, National Wildlife Areas, Migratory Bird Sanctuaries, Great Bear Rainforest schedules, Flathead watershed) are downloaded from federal or non-BCGW repositories and stored locally in `source_data/`. *Federal datasets are excluded by default but have an option to be included.

All source data is reprojected to NAD 1983 BC Environment Albers (EPSG:3005) and loaded into a working File Geodatabase. Per-source preprocessing (spatial clips, attribute-based dissolves) is applied where required, as defined in the CSV configuration.

Priority System

Each designation is assigned a `process_order` — a positive integer that establishes its **priority** relative to all other designations. A **lower** `process_order` indicates **higher** priority. The ordering was carried over from the original Land Designations script and reflects the relative stringency and legal standing of each designation:

Priority tier	process_order range	Examples
Highest	1–6	National Parks, Ecological Reserves, Provincial Parks, Conservancies, Protected Areas, Recreation Areas
High	7–15	Wildlife Management Areas, National Wildlife Areas, Mineral Reserves, Ungulate Winter Range (No Harvest)
Medium	16–31	Wildland Areas, Mining & Tourism Areas, Old Growth Management Areas, VQO Preserves/Retention, UWR/ WHA Conditional Harvest
Low	32–42	

Priority tier	process_order range	Examples
		VQO Partial Retention/Modify/Max Modify, Community Watersheds, Fisheries Sensitive Watersheds, Great Bear EBM Areas, Haida Gwaii EBM Areas

This ordering is maintained in `sources_designations.csv` and is the primary mechanism for resolving spatial overlaps in the planarized output.

Restriction Classification

Each designation carries an analyst-defined restriction rating for three resource industries: **forestry**, **oil & gas**, and **mining**. These ratings are based on the legislation, regulation, or management plan governing each designation and are expressed on a six-level ordinal scale:

Level	Integer code	Interpretation
Protected	5	Fully protected from industrial activity by statute
Full	4	Full restriction — industrial activity prohibited under current management framework
High	3	High restriction — activity may be permitted under limited, tightly controlled conditions
Medium	2	Moderate restriction — activity generally requires special approval or conditions
Low	1	Low restriction — activity is broadly permissible with standard regulatory requirements
None	0	No restriction imposed by this designation

The restriction values are maintained as text labels in the source CSV and converted to integer codes at runtime. They are **not** derived from any external GIS layer or database — they represent an analytical judgment by the project team based on policy review.

Overlapping Output

The first analytical product, `designations_overlapping`, is produced by clipping each of the 42 designation layers to British Columbia's terrestrial and marine boundary and inserting them into a single feature class. Overlaps between different designations are **preserved** — a single geographic area may carry attributes from multiple designation polygons stacked on top of each other. This output is suitable for queries such as *"list all designations that apply to this parcel."*

Planarized Output

The second analytical product, `designations_planarized`, resolves all overlaps to produce a **non-overlapping (planar)** layer. The method is:

1. **Union:** All polygons from `designations_overlapping` are passed through an ArcGIS `Union` operation. This splits every polygon at every intersection boundary, creating planar topology. Where n designations overlap, the Union produces n rows sharing geometrically identical polygon fragments — each row carrying the attributes of one contributing designation.
2. **Fragment grouping:** The Union fragments are grouped by spatial identity using a composite key of centroid coordinates (X and Y to 7 decimal places, $\approx$ 0.01 m precision in BC Albers), polygon area (2 decimal places), and polygon perimeter (2 decimal places). This key uniquely identifies each geometrically distinct fragment without modifying or simplifying the geometry — all original polygon boundaries from the Union are carried forward unchanged, preserving area accuracy.
3. **Priority resolution:** For each group of spatially identical fragments:
 - The **designation** is assigned from the fragment with the **lowest process_order** (highest priority). For example, where a Provincial Park (process_order 3) overlaps an Old Growth Management Area (process_order 19), the polygon is attributed as `park_provincial`.
 - The **restriction values** are set to the **maximum** across all overlapping designations for each industry, so the most restrictive rating always prevails. For example, if one overlapping designation has `forest_restriction = 3` (High) and another has `forest_restriction = 5` (Protected), the output polygon receives `forest_restriction_max = 5`.
 - An **overlapping_designations** field records the codes of **all** contributing designations as a semicolon-delimited string, sorted by `process_order` (e.g., `park_provincial; ogma_legal; vqo_retain`). This preserves the information about which designations were present before priority resolution collapsed them to a single attribution.
4. **Output generation:** The grouped results are written to the `designations_planarized` feature class using an `InsertCursor`, with a spatial index built for query performance.

The planarized output guarantees that every point in BC falls within **at most one** designation polygon, making it suitable for area-based reporting, cartographic display, and industry-sector restriction mapping.

Federal Exclusion

Three of the 42 designation layers originate from **federal** jurisdiction (National Parks, National Wildlife Areas, Migratory Bird Sanctuaries). Because this analysis focuses on **provincially-managed** lands and the federal sources are drawn from a separate data repository with different update cycles, these layers are excluded by default. The `jurisdiction` column in the source CSV identifies federal sources, and the `--exclude-federal` / `--no-exclude-federal` flag controls their inclusion at runtime, alternatively default settings can be hardcoded at the top of `main.py`. Excluding federal sources does not affect the `process_order` numbering of remaining layers — the original ordinal values are preserved to maintain consistency across runs.

Date-Based Change Detection

A date-filter mode (`--recent-only`) restricts the analysis to designations established or modified within a specified time window, directly supporting the 180-day SARA reporting cycle. Each source row in the CSV defines a `date_filter_query` template (e.g., `ESTABLISHMENT_DATE >= '{start_date}'`) that is injected into the WFS CQL filter at download time. Sources without a date-filterable attribute (noted as *"no date field available"* or *"non-BCGW source"*) are excluded from date-filtered runs. An `xlsx` report is generated summarising the changes, excluded layers, feature counts, and pipeline options used.

When the date filter is active, all output feature class and table names are automatically suffixed with `_date_filter` (e.g., `designations_overlapping_date_filter`, `designations_planarized_date_filter`, `cha_overlap_summary_date_filter_03_20`) so that date-filtered outputs are immediately distinguishable from full-run outputs in the geodatabase.

Limitations

- **Restriction ratings are analyst-defined** — they represent informed professional judgment based on policy review, not a legally authoritative determination. Users should consult the underlying legislation and management plans for site-specific decisions.
- **Temporal snapshot** — the dataset reflects the state of BCGW data at the time of download. Designations may have been added, modified, or removed since the last run.
- **Spatial precision** — the fragment-grouping method uses high-precision rounding (7 decimal places for centroid coordinates in BC Albers) to identify geometrically identical Union fragments. In extremely rare edge cases, two genuinely different polygons with nearly identical centroids, areas, and perimeters could theoretically be grouped together; however, this has not been observed in practice with the current source data.

- **Marine extent** — the analysis includes marine areas (marine ecosections and ABMS boundary) in the clipping boundary, so some designations may extend offshore. Users interested in terrestrial-only results should apply a post-processing clip to the land boundary.

Geoprocessing Overview

What the pipeline produces

The pipeline combines roughly 40 provincial designation layers (parks, conservancies, wildlife management areas, old-growth management areas, etc.) into **two output feature classes**, both projected in **BC Albers (EPSG:3005)**:

Output	Description
designations_overlapping	Every designation polygon clipped to BC's terrestrial boundary and stacked. Polygons from different sources can overlap — a single geographic area may carry attributes from multiple designations.
designations_planarized	A non-overlapping (planar) layer derived from the overlapping output. Where designations overlap, the polygon is assigned to the designation with the lowest process_order (highest priority). Adjacent polygons of the same designation are dissolved together.

Coordinate system

All processing is performed in **NAD 1983 BC Environment Albers (EPSG:3005)**, the standard provincial projection for BC government spatial analysis.

Data sources

- **BC Geographic Warehouse (BCGW)**: Most designation layers are downloaded automatically via the province's public WFS endpoint (<https://openmaps.gov.bc.ca/geo/pub/wfs>). The pipeline resolves BC Data Catalogue URLs to WFS layer names and fetches GeoJSON features with optional CQL query filters.
- **Manual downloads**: A few sources (e.g., private conservation lands) are not available via WFS. These are placed in the `source_data/` folder and referenced from the CSV.

Federal exclusion

Three designation layers originate from federal jurisdiction:

process_order	Designation
1	National Parks (Administered Lands)
10	National Wildlife Areas
12	Migratory Bird Sanctuaries

By default these are **excluded** (`--exclude-federal`, enabled by default) because this analysis focuses on provincially-managed lands. Use `--no-exclude-federal` to include them.

Restriction levels

Each designation carries restriction ratings for three resource industries (`forest_restriction`, `og_restriction`, `mine_restriction`). These values are **analyst-defined classifications** maintained as text labels in `sources_designations.csv` — each of the 42 source rows specifies a restriction level for forestry, oil & gas, and mining based on the policy or legislation governing that designation. At runtime, `designatedlands.py` converts the text labels to integer codes using a lookup dictionary defined in the `DesignatedLands.__init__()` constructor:

Level	Code	Meaning
Protected	5	Fully protected from industrial activity
Full	4	Full restriction on the given industry
High	3	High restriction
Medium	2	Medium restriction
Low	1	Low restriction
None	0	No restriction

No external GIS layer or database table defines these values — they originate entirely from the CSV and are encoded into the output feature class attributes during the `create_designations_overlapping()` step.

In the planarized output, where multiple designations overlap, the **maximum** restriction value for each industry is retained (highest restriction wins).

Geoprocessing Workflow

The pipeline runs seven sequential steps. Each step builds on the outputs of the previous one.

1. TEST CONNECTION	Verify working GDB is accessible
2. DOWNLOAD	Fetch each layer from BCGW WFS → GDB FCs (skip layers already present in GDB)
3. PREPROCESS	Clip / dissolve per-source as needed; merge land + marine → bc_boundary
4. PROCESS VECTOR	4a. designations_overlapping (clip & stack) 4b. designations_planarized (Union→Dissolve)
5. PROCESS RASTER	(Optional) PolygonToRaster → overlay TIFs
6. DUMP Geodatabase	Export FCs → output File
7. CLEANUP	Delete intermediate src_* / *_pp FCs

Step 1 — Test Connection

Verifies the working File Geodatabase (`designatedlands.gdb`) is accessible and lists the feature classes currently in it. Catches connection problems before spending time on downloads.

Step 2 — Download

For each source row in the CSV:

1. **Resolve** the BC Data Catalogue URL to a WFS layer name (e.g., `WHSE_TANTALIS.TA_PARK_ECORES_PA_SVW`).
2. **Fetch** features from the BCGW WFS endpoint as GeoJSON, applying any CQL query filter defined in the CSV.
3. **Convert** the GeoJSON to a feature class in the working GDB using `arcpy.conversion.JSONToFeatures`.
4. **Name** the feature class using the stable `process_order` value from the CSV (e.g., `src_02_park_er`), so that re-runs with different filter flags still match existing FCs and skip re-downloading.

For manual sources (`manual_download = T`), the data is loaded from local files in `source_data/` using `arcpy.conversion.FeatureClassToFeatureClass`.

Supporting layers (BCGS 1:20k tiles, BC boundary polygons, marine ecosections) are also downloaded here.

Step 3 — Preprocess

Applies per-source preprocessing operations defined in the CSV:

- **Clip**: Clip a source FC by another FC (e.g., clip NGO conservation lands by `mk_boundary` to restrict to the Muskwa-Kechika Management Area).
- **Dissolve** (union): Dissolve overlapping features within a single source by specified attribute columns (e.g., dissolve multiple overlapping conservation land parcels by `CONSERVATION_LAND_TYPE`).

Preprocessed FCs are saved with a `_pp` suffix (e.g., `src_05_ngo_land_pp`).

Also creates the `bc_boundary` feature class by merging:

- `bc_boundary_land` (provincial terrestrial boundary)
- Marine areas from `bc_abms` (BC Boundary ABMS) and `marine_ecosections`

The merged boundary is dissolved into a single polygon representing BC's full land and marine extent.

Step 4 — Process Vector

4a. `designations_overlapping`

Creates an empty feature class with fields: `process_order`, `designation`, `source_id`, `source_name`, `forest_restriction`, `og_restriction`, `mine_restriction`. Then iterates each source in `process_order`:

1. Selects the preprocessed FC (if available) or the raw source FC.
2. **Clips** it to `bc_boundary` (land boundary) using `arcpy.analysis.Clip`.
3. Opens a `SearchCursor` on the clipped result and an `InsertCursor` on the output.
4. For each feature, inserts a row with the designation attributes and restriction levels looked up from the CSV configuration.

The result is a single FC where all designation polygons are stacked — overlaps between different designations are preserved.

4b. `designations_planarized`

Takes `designations_overlapping` and produces a non-overlapping output:

1. **Union** (`arcpy.analysis.Union`): Splits all polygons at every intersection boundary, creating planar topology. Every resulting polygon fragment knows which original designations it belonged to.
2. **Python-based spatial grouping** similar to "dissolve": Fragments are grouped by a composite key of centroid coordinates, area, and perimeter (not an `arcpy.management.Dissolve` call). For each group of spatially identical fragments:
 - The designation with the **lowest process_order** (highest priority) is assigned.
 - `forest_restriction_max`, `og_restriction_max`, and `mine_restriction_max` are set to the MAX across all contributing designations, ensuring the most restrictive level is reported for each sector.
 - All contributing designation names are collected into a semicolon-delimited `overlapping_designations` field.
3. Populates the output using `InsertCursor`, writing each unique planar polygon with its aggregated attributes. Each Union fragment retains its original geometry — polygons are not geometrically merged.

The result: every point in BC falls within at most one designation polygon, assigned to the highest-priority overlapping designation.

Step 5 — Process Raster (Optional)

Requires the **Spatial Analyst** extension (not available with ArcGIS Pro Basic). Disabled by default (`--raster` to enable).

- **Rasterize**: Converts each designation source to a GeoTIFF at the configured resolution (default 100m) using `arcpy.conversion.PolygonToRaster`.
- **Overlay**: Uses NumPy array operations to combine all rasters into four outputs:
 - `designatedlands.tif` — designation codes (highest process_order wins)
 - `forest_restriction.tif` — forest restriction levels
 - `og_restriction.tif` — oil & gas restriction levels
 - `mine_restriction.tif` — mine restriction levels

Step 6 — Dump

Exports `designations_overlapping` and `designations_planarized` from the working GDB into a clean output File Geodatabase at `outputs/designatedlands_output.gdb`. Creates the output GDB if it doesn't exist; overwrites existing FCs if they do.

Step 7 — Cleanup

Deletes all intermediate feature classes (`src_*` and `*_pp`) from the working GDB to reclaim disk space. The output GDB in `outputs/` is not affected. Use `--skip-cleanup` to keep intermediate data for debugging.

Requirements

- **ArcGIS Pro** (tested with ArcGIS Pro 3.x)
- **ArcGIS Pro Basic** license (minimum) — Spatial Analyst extension needed only for raster processing
- **Python 3.9+** (via `arcgispro-py3` conda environment)
- **arcpy** (included with ArcGIS Pro)
- **openpyxl** 3.1.2+ (for xlsx report generation)
- Network access to `openmaps.gov.bc.ca` (BCGW WFS)

Installation

1. Clone the repository:

```
git clone https://github.com/cjsostad/designatedlands---AG.git
cd "designatedlands - AG"
```

2. Activate the ArcGIS Pro Python environment:

```
conda activate arcgispro-py3
```

3. Install openpyxl (if not already available):

```
pip install openpyxl
```

4. If any data sources are marked `manual_download = T` in the source CSV, download those datasets to the `source_data/` folder.

Usage

Full pipeline (recommended)

Run the entire pipeline with default settings (no arguments required):

```
python main.py
```

This will:

- Exclude federal layers (National Parks, NWA, MBS)
- Apply the full CHA definition query (FINAL + BC + species exclusions)
- Skip raster processing
- Download all layers, preprocess, build vector outputs, export, and clean up

Common options

```
# Include federal layers
python main.py --no-exclude-federal

# Enable raster processing (requires Spatial Analyst)
python main.py --raster

# Use a specific config file
python main.py --config config_2020-10-08.cfg

# Filter to recently changed designations only
python main.py --recent-only --start-date 2025-04-01

# Keep intermediate data for inspection
python main.py --skip-cleanup

# Skip downloads (use what's already in the GDB)
python main.py --skip-download

# Verbose logging
python main.py --verbose
```

All command-line flags

Flag	Default	Description
<code>--config, -c</code>	None	Path to <code>.cfg</code> configuration file
<code>--verbose, -v</code>	off	Increase log verbosity
<code>--quiet, -q</code>	off	Suppress log output
<code>--skip-download</code>	off	Skip the download step
<code>--raster / --no-raster</code>	off	Enable/disable raster processing
<code>--skip-cleanup</code>	off	Keep intermediate FCs in working GDB
<code>--recent-only</code>	off	Filter sources to date window
<code>--start-date</code>	2025-04-01	Start of date filter window
<code>--end-date</code>	today	End of date filter window

Flag	Default	Description
<code>--exclude-federal</code> / <code>--no-exclude-federal</code>	on	Exclude/include federal designations

The following option is set directly in `main.py` (not as a CLI flag):

Variable	Default	Description
<code>CHA_FILTER_OUT_WRS</code>	True	True = full CHA filter (FINAL + BC + exclude specified wide ranging species). False = minimal CHA filter (FINAL + BC only, all species included).

Subcommand interface

`designatedlands.py` also exposes individual pipeline steps as subcommands for advanced use:

```
python designatedlands.py download [CONFIG_FILE]
python designatedlands.py preprocess [CONFIG_FILE]
python designatedlands.py process-vector [CONFIG_FILE]
python designatedlands.py process-raster [CONFIG_FILE]
python designatedlands.py dump [CONFIG_FILE]
python designatedlands.py cleanup [CONFIG_FILE]
python designatedlands.py overlay IN_FILE OUT_FILE [CONFIG_FILE]
```

Smart resume

`resume_pipeline.py` automatically detects which pipeline steps have already completed (by inspecting the GDB contents) and resumes from where the last run stopped:

```
python resume_pipeline.py
```

Configuration

An INI-format `.cfg` file can be supplied via `--config`. Only include parameters you want to override — defaults are used for the rest.

See [designatedlands_sample_config.cfg](#) for all available settings:

Key	Default	Description
<code>dl_path</code>	<code>source_data</code>	Folder for downloaded source data

Key	Default	Description
<code>sources_designations</code>	<code>sources_designations.csv</code>	Designation source definitions
<code>sources_supporting</code>	<code>sources_supporting.csv</code>	Supporting layer definitions
<code>out_path</code>	<code>outputs</code>	Output folder for final GDB and reports
<code>gdb_path</code>	<code>designatedlands.gdb</code>	Working File Geodatabase
<code>resolution</code>	100	Raster output resolution in metres
<code>n_processes</code>	<code>multiprocessing.cpu_count() - 1</code>	Parallel processes for raster overlay

Source CSV Files

`sources_designations.csv`

Defines all 42 designation layers. Each row configures one data source. Key columns:

Column	Description
process_order	Integer defining overlay priority. Lower number = higher priority.
exclude	T to exclude the source from all operations.
manual_download	T if the source must be downloaded manually to <code>source_data/</code> .
name	Full name of the designation (e.g., "National Parks (Administered Lands)").
jurisdiction	<code>federal</code> for federal sources; blank for provincial. Used by <code>--exclude-federal</code> .
designation	Machine-readable underscore-separated code (e.g., <code>park_national</code>).
source_id_col	Column in the source data providing the unique feature ID.
source_name_col	Column providing the feature name.
forest_restriction	Restriction level for forestry: <code>Protected</code> , <code>Full</code> , <code>High</code> , <code>Medium</code> , <code>Low</code> , <code>None</code> .

Column	Description
og_restriction	Restriction level for oil & gas.
mine_restriction	Restriction level for mining.
url	BC Data Catalogue URL or direct download URL.
bcgw_layer_name	BCGW WFS layer name (if different from catalogue-resolved name).
query	CQL filter for WFS requests (e.g., <code>PARK_CLASS <> 'REC'</code>).
date_filter_query	CQL query with <code>{start_date}</code> / <code>{end_date}</code> placeholders for <code>--recent-only</code> .
preprocess_operation	<code>clip</code> or <code>union</code> (dissolve).
preprocess_args	Arguments for preprocessing (clip boundary FC or dissolve columns).

[sources_supporting.csv](#)

Defines 7 supporting layers used during processing (not designation layers themselves):

- **BCGS 1:20k Grid** (`tiles_20k`) — tile index for parallel processing
- **NTS 250k Grid** (`tiles_250k`) — national topographic tile index
- **BC Boundary ABMS** (`bc_abms`) — administrative boundary (marine)
- **BC Boundary Land** (`bc_boundary_land`) — terrestrial boundary
- **Marine Ecosections** (`marine_ecosections`) — marine ecological zones
- **Muskwa-Kechika Boundary** (`mk_boundary`) — management area for clipping
- **Critical Habitat Area** (`critical_habitat_area`) — ECCC's Critical Habitat polygons (filtered to FINAL status, British Columbia, terrestrial species) used for the CHA intersection step

Code Breakdown

Project structure

```

└─ main.py                      # Full pipeline runner
(recommended entry point)
└─ designatedlands.py          # Core DesignatedLands class
and geoprocessing logic
└─ date_filter.py              # Date-based filtering and xlsx
report generation
└─ create_cha.py                # Download and prepare the CHA
dataset from ECCC
└─ Create_CHA_AOI.py           # CHA preparation with AOI clip

```

```

(for testing)
├─ intersect_area_calc.py          # CHA × designation
intersection and area calculations
├─ find_schema.py                 # Query BCGW WFS for date/time
fields in source schemas
├─ _test_all_cql.py               # Validate all CQL queries
against WFS endpoints
├─ sources_designations.csv       # 42 designation source
definitions
├─ sources_supporting.csv         # 7 supporting layer
definitions (incl. CHA)
├─ designatedlands_sample_config.cfg # Example configuration file
├─ designatedlands.gdb/           # Working File Geodatabase
(intermediate data)
├─ source_data/                  # Downloaded / manual source
data files
├─ outputs/                      # Output GDB and reports
│   └─ designatedlands_output.gdb/ # Final clean output
geodatabase
│   └─ designated_lands_pipeline_report.xlsx # Pipeline report
├─ logs/                         # Timestamped run logs
├─ rasters/                      # Raster outputs (when --raster
is used)
└─ scripts/                     # Utility scripts
    └─ utility_scripts/
        └─ resume_pipeline.py      # Smart resume with auto-
detection of completed steps
            └─ run_planarized.py   # Standalone planarized output
runner

```

`main.py` — Pipeline Orchestrator

The recommended entry point. Parses command-line arguments, initialises the `DesignatedLands` object, and runs each pipeline step in sequence with error handling and progress output.

Key responsibilities:

- Prints a configuration banner showing active flags
- Calls `DesignatedLands()` constructor (loads CSVs, applies filters, sets up GDB)
- Wraps each step in `run_step()` which captures `arcpy` messages and logs failures
- Generates an `xlsx` report when federal exclusion or date filtering is active
- Controls optional steps (raster, cleanup) via CLI flags
- Controls the CHA definition query scope via `CHA_FILTER_OUT_WRS` — when `False`, overrides the CSV query to include all species (FINAL + BC only)

designatedlands.py — Core Processing Engine

Contains the `DesignatedLands` class with all geoprocessing methods:

Method	Purpose
<code>__init__()</code>	Load config, read CSVs, filter sources, set up GDB and arcpy environment
<code>_read_sources()</code>	Parse CSV, validate process_order, apply federal exclusion & date filtering
<code>_validate_sources()</code>	Check that process_order starts at 1 and has no gaps or duplicates
<code>download()</code>	Fetch all sources from WFS or local files into the working GDB
<code>preprocess()</code>	Apply per-source clip/dissolve operations
<code>create_bc_boundary()</code>	Merge land + marine boundaries into single bc_boundary FC
<code>create_designations_overlapping()</code>	Clip each source to BC and stack into overlapping output
<code>create_designations_planarized()</code>	Union → priority assignment → Dissolve into non-overlapping output
<code>rasterize()</code>	Convert vector designations to per-source GeoTIFFs
<code>overlay_rasters()</code>	Combine rasters using NumPy (highest priority wins)
<code>dump()</code>	Export final FCs to <code>outputs/designatedlands_output.gdb</code>
<code>overlay()</code>	Intersect an external layer with <code>designations_overlapping</code>
<code>cleanup()</code>	Remove intermediate <code>src_*</code> and <code>*_pp</code> FCs from working GDB
<code>test_connection()</code>	Verify GDB accessibility

The module also provides:

- **Automatic arcpy message logging:** All arcpy tool calls are wrapped to log geoprocessing messages at the appropriate level.
- **WFS download infrastructure:** Functions to resolve catalogue URLs, fetch GeoJSON via WFS, strip Z-coordinates, and load results into a GDB.

- **Raster attribute table creation:** `create_rat()` function for building `.tif.vat.dbf` files.

`date_filter.py` — Reporting & Date Filtering

Provides date-based WFS queries and xlsx report generation using **openpyxl**:

- `apply_date_filter_to_query()` — Injects `{start_date}` and `{end_date}` into CQL query templates.
- `run_report()` — Queries WFS for recently changed features and generates a 4-sheet Excel workbook:
 - **Changes:** Features added/modified within the date window
 - **Excluded Layers:** Layers removed by date or federal filter
 - **Summary:** Feature counts per designation
 - **Pipeline Options:** Flags used for the current run
- `write_report_xlsx()` — Low-level workbook creation with formatted headers and auto-sized columns.

`resume_pipeline.py` — Smart Resume

`detect_completed_steps()` inspects the working GDB to determine which pipeline steps have already been completed:

- Checks for `src_*` feature classes matching the expected source list
- Checks for `bc_boundary` existence
- Checks for `designations_overlapping` and `designations_planarized`
- Checks for FCs inside `outputs/designatedlands_output.gdb`
- Checks whether intermediate FCs have been cleaned up

Resumes from the first incomplete step, or can be overridden with

`--force-from STEP`.

`create_cha.py` — CHA Dataset Preparation

Downloads and prepares the **Critical Habitat Area (CHA)** dataset from ECCC's national data portal:

- Reads the CHA entry from `sources_supporting.csv` (URL, definition query, field mappings).
- Downloads `CriticalHabitat.zip` with retry logic (up to 3 attempts) and extracts the geodatabase into `source_data/`.
- Applies the definition query to filter to **FINAL** status, **British Columbia** province, and (by default) excludes specified species. When called with `query_override`, uses the provided query instead of the CSV-defined one.
- Falls back to an existing local copy of `CriticalHabitat.gdb` if all download attempts fail.

Can be run standalone (`python create_cha.py`) or called from the pipeline via `prepare_cha()`.

`Create_CHA_AOI.py` — CHA with AOI Clip (Testing)

A testing variant of `create_cha.py` that adds an **Area of Interest (AOI)** clip. Filters the CHA dataset to a spatial extent for faster development iterations, producing a smaller feature class suitable for debugging the intersection workflow without processing the full provincial dataset.

`intersect_area_calc.py` — CHA Intersection & Area Calculations

Performs the spatial intersection between designated lands and CHA, then quantifies protection coverage:

- `run_cha_intersection()` — Master function that intersects both `designations_planarized` and `designations_overlapping` with the CHA feature class.
- Calculates geodesic overlap areas (hectares) and produces two summary tables:
 - `cha_overlap_summary` — Per-designation summary: what percentage of each designation overlaps with CHA.
 - `cha_protection_summary` — Per-CHA-polygon summary: what percentage of each CHA polygon is covered by designated lands.
- Adds `CHA_Protected_Pct` to each intersect feature for per-fragment protection percentages.

Can be run standalone or imported as a module from the pipeline.

`find_schema.py` — WFS Schema Inspector

Queries the BCGW WFS `DescribeFeatureType` endpoint for each layer referenced in the source CSVs and reports any fields whose XML schema type contains `date` or `time`. Used during initial configuration to identify which source layers support date-based filtering.

`_test_all_cql.py` — CQL Query Validator

Validates every CQL query defined in the source CSVs against the live WFS endpoint:

1. Checks that all field names referenced in CQL filters exist in the WFS schema (via `DescribeFeatureType`).
2. Sends each CQL filter to the WFS endpoint and verifies it returns HTTP 200.

Run before production pipeline executions to catch broken or outdated CQL queries early.

Vector Outputs

The `dump` step writes two feature classes to `outputs/designatedlands_output.gdb`:

Naming convention: When the date filter is active (`--recent-only`), all output names are suffixed with `_date_filter` (e.g., `designations_overlapping_date_filter`, `designations_planarized_date_filter`). This ensures date-filtered outputs are immediately distinguishable from full-run outputs when both exist in the same geodatabase.

`designations_overlapping`

Each individual designation polygon, clipped to land boundary, with full attribution. Overlaps are preserved.

Field	Description
<code>process_order</code>	Priority rank from CSV
<code>designation</code>	Machine-readable designation code
<code>source_id</code>	Original feature ID from source data
<code>source_name</code>	Original feature name from source data
<code>forest_restriction</code>	Forestry restriction level (0–5)
<code>og_restriction</code>	Oil & gas restriction level (0–5)
<code>mine_restriction</code>	Mining restriction level (0–5)

`designations_planarized`

Non-overlapping output. Where designations overlap, the polygon is assigned to the highest-priority designation. Restriction fields hold the maximum value across all overlapping designations. All contributing designation names are listed in `overlapping_designations`.

Field	Description
<code>designation</code>	Highest-priority designation code (lowest <code>process_order</code>)
<code>overlapping_designations</code>	Semicolon-delimited list of all designation codes that overlap this polygon (e.g., <code>vqo_retain; fsw</code>), sorted by priority
<code>forest_restriction_max</code>	

Field	Description
	Maximum forest restriction across overlapping designations
<code>og_restriction_max</code>	Maximum oil & gas restriction
<code>mine_restriction_max</code>	Maximum mine restriction

CHA Intersection Outputs

When the CHA (Critical Habitat Area) intersection step runs, it produces two intersect feature classes and two summary tables in the output geodatabase. These outputs answer two distinct questions about the relationship between designated lands and Critical Habitat Areas.

Naming convention: CHA output names include a date stamp (`_MM_DD`). When the date filter is active, `_date_filter` is inserted before the `_cha` segment (e.g., `designations_planarized_date_filter_cha_03_20`, `cha_overlap_summary_date_filter_03_20`).

`cha_overlap_summary`

A **per-designation** summary table that answers: *"What percentage of each designation overlaps with Critical Habitat Area?"*

Built by intersecting `designations_overlapping` with the CHA feature class, then aggregating areas by designation. The key fields are:

Field	Description
<code>designation</code>	The designation code (e.g., <code>park_provincial</code> , <code>ogma_legal</code>)
<code>FREQUENCY</code>	The number of intersect fragments that were aggregated into this designation's summary row (auto-generated by <code>arcpy.analysis.Statistics</code>).
<code>SUM_Overlap_Area_ha</code>	Total area (hectares) of this designation that falls inside CHA polygons. Computed by running geodesic area calculation on the intersect result (overlapping designations clipped to CHA), then summing by designation.
<code>SUM_Total_Area_ha</code>	Total area (hectares) of this designation overall (before intersection with CHA). Computed from the original <code>designations_overlapping</code> layer and joined in.
<code>CHA_Percent</code>	The percentage of the designation's total area that overlaps with CHA.

The percentage is calculated as:

$$\text{CHA_Percent} = \frac{\text{SUM_Overlap_Area_ha}}{\text{SUM_Total_Area_ha}} \times 100$$

How to read it: Each row represents one designation. A `CHA_Percent` of 12.5 means that 12.5% of that designation's total provincial area falls within Critical Habitat Areas. A designation with no CHA overlap will not appear in this table.

`cha_protection_summary`

A **per-CHA-polygon** summary table that answers: "*What percentage of each individual CHA polygon is covered by overlapping designations?*"

Built by grouping the intersect result by `FID_critical_habitat_area` (the CHA polygon ID carried through from the intersection), then summing the overlap area and comparing it to the original CHA polygon area. The key fields are:

Field	Description
<code>FID_critical_habitat_area</code>	The unique identifier for each CHA polygon. This field is auto-generated by <code>arcpy.analysis.PairwiseIntersect</code> — it carries the original <code>OBJECTID</code> of the CHA polygon from the <code>critical_habitat_area</code> feature class into the intersect output, allowing each fragment to be traced back to its parent CHA polygon.
<code>FREQUENCY</code>	The number of intersect fragments that were aggregated into this CHA polygon's summary row (auto-generated by <code>arcpy.analysis.Statistics</code>).
<code>SUM_Overlap_Area_ha</code>	Total area (hectares) of all designation overlaps within this CHA polygon
<code>FIRST_Area_ha</code>	The original area (hectares) of this CHA polygon (from the <code>Area_ha</code> field in the CHA source)
<code>Total_CHA_Protected_Pct</code>	The percentage of this CHA polygon that is covered by designated lands

The percentage is calculated as:

$$\text{Total_CHA_Protected_Pct} = \frac{\text{SUM_Overlap_Area_ha}}{\text{FIRST_Area_ha}} \times 100$$

How to read it: Each row represents one CHA polygon. A

`Total_CHA_Protected_Pct` of 85.0 means that 85% of that particular CHA polygon's area is covered by one or more designated land designations. Values are capped at 100%.

Per-feature `CHA_Protected_Pct`

In addition to the summary tables, the intersect feature classes (`designations_planarized_cha` and `designations_overlapping_cha`) each contain a `CHA_Protected_Pct` field on every individual feature. This shows what fraction of the original CHA polygon is represented by that specific intersect fragment:

$$\text{CHA_Protected_Pct} = \frac{\text{Overlap_Area_ha}}{\text{Area_ha}} \times 100$$

How CHA polygons are fragmented

The original CHA polygons enter the pipeline as whole, unfragmented geometries from ECCC's national dataset. During the `PairwiseIntersect` step in `intersect_area_calc.py`, each CHA polygon is split into smaller fragments wherever it crosses a designation polygon boundary. For example:

- **Input:** 1 CHA polygon (500 ha) overlapping 3 designation areas (provincial park, OGMA, wildlife habitat area).
- **After `PairwiseIntersect`:** 3 fragment rows (e.g., 180 ha + 220 ha + 100 ha = 500 ha total). Each fragment inherits:
 - `Area_ha` = 500 — the **original** CHA polygon area, carried through as an attribute (unchanged).
 - `FID_critical_habitat_area` — the original CHA `OBJECTID` (same value on all 3 fragments).
 - `Overlap_Area_ha` — the fragment's actual geometry area (180, 220, or 100 ha respectively).
 - `designation`, `process_order`, restriction fields from whichever designation the fragment falls within.
- **Per-fragment percentage:** `CHA_Protected_Pct` = `Overlap_Area_ha` / `Area_ha` × 100 (e.g., 180 / 500 = 36%).

The summary tables then **re-aggregate** fragments back to the original CHA polygon level using `FID_critical_habitat_area` as the grouping key. `FIRST(Area_ha)` recovers the original polygon area (since all fragments carry the same value), and `SUM(Overlap_Area_ha)` totals the fragments to compute `Total_CHA_Protected_Pct`.

Raster Outputs (Optional)

When `--raster` is enabled, four GeoTIFFs are produced in `outputs/`:

1. `designatedlands.tif` — Designation codes (highest process_order wins in overlaps)
2. `forest_restriction.tif` — Forest restriction levels
3. `og_restriction.tif` — Oil & gas restriction levels
4. `mine_restriction.tif` — Mine restriction levels

Each raster includes an attribute table (`.tif.vat.dbf`).

License

Copyright 2022 Province of British Columbia

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
implied.

See the License for the specific language governing permissions and
limitations under the License.

This repository was originally posted and maintained by [Environmental Reporting BC](#). Click [here](#) for the original Designated Lands Repository.

Click [here](#) for the repository that contains the script described in this document.